

dbt Training — Trainer Guide

This document is for you, the trainer. It covers what participants need to do per exercise, the expected outcome at every step, common mistakes to watch for, and how to verify success.

Participant-facing document: [exercises.md](#) **Data reference:** [seeds/](#) (Bronze) and [reference/](#) (Silver expected output)

First: demonstration of dbt in snowflake

Done by instructor only

Contains setup, config of [project.yml](#), run in dev, prod, check folders, locate compiled code and see results.

Steps:

- open Snowflake
- create new workspace
- create new dbt project, select warehouse, schema (explain that is fallback)
- edit profiles.yml
 - explain what they see
 - add user: your.email@bloomwell.de
 - change database/schema: bloomwell_staging.dev_schema
 - add prod environment bloomwell_staging.prod_schema
- run it in dev
 - look at output
 - look at results in compiled -> a sql file
 - look at database tables
 - look at the DAG
 - look at documentation in hubspot-tickets-tickets

Pre-training Setup (Snowflake — Trainer Only)

Before training begins, load the following tables into [DBT_TRAINING.HUBSPOT_RAW](#) in Snowflake. Participants do **not** run [dbt seed](#) for these — they are pre-loaded Bronze tables that simulate a production data pipeline.

File	Target table	Schema
seeds/raw_contacts.csv	contacts	DBT_TRAINING.HUBSPOT_RAW
seeds/raw_deals.csv	deals	DBT_TRAINING.HUBSPOT_RAW
seeds/raw_pipeline_stages.csv	pipeline_stages	DBT_TRAINING.HUBSPOT_RAW
seeds/raw_prescriptions.csv	prescriptions	DBT_TRAINING.HUBSPOT_RAW
seeds/raw_products.csv	products	DBT_TRAINING.HUBSPOT_RAW
seeds/raw_owners.csv	owners	DBT_TRAINING.HUBSPOT_RAW

Quick load command (Snowflake worksheet):

```
-- Run once per training session before participants connect
CREATE SCHEMA IF NOT EXISTS DBT_TRAINING.HUBSPOT_RAW;
-- Then use Snowflake's "Load Data" UI or SnowSQL COPY INTO to load each CSV file
```

Participant environment variables to set before `dbt debug`:

```
export DBT_SNOWFLAKE_ACCOUNT=<account_locator>
export DBT_SNOWFLAKE_USER=<user>
export DBT_SNOWFLAKE_PASSWORD=<password>
export DBT_USER_NAME=<lastname>           # dev schema becomes DEV_<lastname>
export DBT_SOURCE_DATABASE=DBT_TRAINING   # points to pre-loaded bronze tables
```

Participants load only the lookup seeds themselves (in Module 08): `dbt seed --select country_codes product_categories`

Deliberate Bugs in the Pre-built Project

Three bugs have been planted in the scaffold — presented as "code submitted by colleagues for review." Participants aren't told about Bug 2 or Bug 3 upfront; tests and data inspection surface them.

Bug	File	What's wrong	When it surfaces
Bug 1 (visible)	<code>models/1_staging/stg_hubspot_pipeline_stages.sql</code>	<code>materialized='table'</code> — staging models must be views	Module 01 Task 3 (spot it); Module 04 Task 1 (fix it)
Bug 2 (hidden)	<code>models/2_silver/fct_prescription.sql</code>	<code>patient_key</code> and <code>doctor_key</code> aliases are reversed	Module 06 Task 2 — FK relationship test fails
Bug 3 (hidden)	<code>models/3_gold/mrt_country_summary.sql</code>	Join uses <code>p.patient_key</code> instead of <code>p.country_code</code> — all counts are 0	Module 08 Task 3 — data inspection reveals zero rows

Bug 2 detail: The SELECT aliases `doc.doctor_key AS patient_key` and `pat.patient_key AS doctor_key` — reversed. The model compiles and runs without error, but `fct_prescription.doctor_key` contains patient surrogate values. The relationship test

`relationships_fct_prescription_doctor_key__doctor_key__ref_dim_doctor_` catches it (contact_ids 6-20 have no matching owner in dim_doctor). The fix is a two-line swap of the aliases.

Bug 3 detail: The Gold mart joins `cc.country_code = p.patient_key` — comparing a two-letter country code to an MD5 hash. Every LEFT JOIN returns NULL, so every country shows `contact_count = 0` and `total_deal_amount = 0`. No test fails — participants catch it by querying `SELECT * FROM mrt_country_summary` and noticing all zeros. The fix: change `p.patient_key` to `p.country_code`.

Teaching point: Tests catch structural bugs (Bug 2). Data inspection catches semantic bugs (Bug 3). Both layers of validation are necessary.

Module 01 — Orientation (25 min)

What participants do

Download repo in their workspace.

- add user, change dev to bloomwell_staging, schema: dev_tweber, dev_a..., dev_lastname
- 1. Read `dbt_project.yml` and answer four questions
- 2. List all `.sql` files in `models/silver/` and find one `{{ ref() }}` or `{{ source() }}` call in each
- 3. Open `stg_hubspot__pipeline_stages.sql` and spot the bug (without fixing it)
- 4. Run `dbt ls`

Expected outcomes

Task	Expected answer
Project name	<code>analytics</code>
Staging materialization	<code>view</code>
Silver materialization	<code>table</code>
<code>+database</code> key	Routes Silver models to a specific Snowflake database instead of the profile default
Silver models	<code>dim_pipeline</code> , <code>dim_patient</code> , <code>dim_doctor</code> , <code>fct_prescription</code>
Bug in pipeline stages	<code>materialized='table'</code> — should be <code>'view'</code>
<code>dbt ls</code> count	5 (4 Silver + 1 staging pre-built)

Common mistakes

- Participants try to fix the bug immediately — redirect them: "note it, fix it in Module 04"
- Participants can't find `dbt_project.yml` — it's at the repo root, not inside `models/`

Trainer notes

The goal here is orientation, not comprehension. If participants struggle with any question, that signals a gap to revisit — not a reason to slow down. The Module 01 theory block should have covered all of this.

Module 02 — Local Setup (no exercise)

Success criterion

```
dbt debug
```

All lines show **OK**. The most common failure is an incorrect **account** field in **profiles.yml** — Snowflake account locators must include the region suffix (e.g., **xy12345.eu-west-1**).

Trainer notes

Don't proceed to Module 03 with anyone whose **dbt debug** is still failing. Snowflake auth with **externalbrowser** requires a browser window — make sure participants aren't in a headless environment.

Module 03 — Write Your First Staging Model (20 min)

What participants do

1. Write down their prediction of how a given model compiles (Task 1 — no tools)
2. Create **models/staging/hubspot/stg_hubspot__contacts.sql** (Task 2)
3. Run **dbt compile --select stg_hubspot__contacts**
4. Open the compiled output file and compare to their prediction

Expected outcome: Task 1

Jinja expression	Compiled to
<code>{{ source('hubspot', 'contacts') }}</code>	<code>BRONZE.HUBSPOT.contacts</code>
<code>{{ ref('dim_pipeline') }}</code>	<code>SILVER_DEV.TESTING__dev_yourname.dim_pipeline (dev target)</code>
<code>{{ config(materialized='view') }}</code>	Removed; becomes a <code>CREATE OR REPLACE VIEW</code> wrapper

This trips people up: participants often write `SILVER.PUBLIC.dim_pipeline` for the `ref()`. On **dev**, `ref()` resolves to the **dev** database and personal schema.

Expected outcome: Task 2

```
{{ config(materialized='view') }}

SELECT
  contact_id,
  email,
  pipeline_id,
  _loaded_at AS loaded_at
FROM {{ source('hubspot', 'contacts') }}
```

Compiled SQL (dev):

```
CREATE OR REPLACE VIEW SILVER_DEV.TESTING__dev_yourname.stg_hubspot__contacts AS (

  SELECT
    contact_id,
    email,
    pipeline_id,
    _loaded_at AS loaded_at
  FROM BRONZE.HUBSPOT.contacts

)
```

Common mistakes

Mistake	Fix
Hardcodes <code>BRONZE.HUBSPOT.contacts</code> instead of <code>{{ source() }}</code>	Explain: if the database name changes, every hardcoded path breaks; source declarations fix it in one place
Uses <code>{{ source('contacts') }}</code> (missing source name)	Must be <code>{{ source('hubspot', 'contacts') }}</code> — two arguments: source block name, then table name
Forgets <code>{{ config(materialized='view') }}</code>	Without it, the default from <code>dbt_project.yml</code> applies — which is also view for staging, but participants should learn to be explicit
Uses <code>SELECT *</code>	Fine for staging in production, but discouraged here — participants should practise naming columns explicitly

Verify success

```
dbt compile --select stg_hubspot__contacts
cat target/compiled/analytics/models/staging/hubspot/stg_hubspot__contacts.sql
```

Confirm `BRONZE.HUBSPOT.contacts` appears in the output.

Bonus — Variables and environment-aware filtering

Participants use an AI assistant to figure this out independently. There's no prescribed solution — the goal is the discovery process, not a specific answer. Valid solutions share these properties:

- A variable declared under `vars:` in `dbt_project.yml`, e.g. `limit_rows: 1000`
- A `WHERE` clause in the model body using `{{ target.name }}` to gate the filter on dev only
- `{{ var('limit_rows') }}` used inside the condition

A correct model looks roughly like:

```
SELECT ...
FROM {{ source('hubspot', 'contacts') }}
{% if target.name == 'dev' %}
WHERE contact_id IN (
```

```
SELECT contact_id FROM {{ source('hubspot', 'contacts') }} LIMIT {{
var('limit_rows') }}
)
{% endif %}
```

Or more simply using `LIMIT` directly:

```
SELECT ...
FROM {{ source('hubspot', 'contacts') }}
{% if target.name == 'dev' %}
LIMIT {{ var('limit_rows') }}
{% endif %}
```

Command-line override to verify:

```
dbt compile --select stg_hubspot__contacts --vars '{"limit_rows": 100}'
```

The compiled SQL should contain `LIMIT 100`. Without `--vars`, `LIMIT 1000` (the default) appears in dev. In prod the `LIMIT` clause disappears entirely.

Module 04 — Code Review Exercise (25 min)

Scenario

A colleague pushed 3 new models and `dbt build` is now broken. Participants check out the exercise branch ([exercises/module-04](#)), run the project, read the code, and identify all 5 bugs.

Setup (trainer only — before the session)

The 3 exercise files live on a separate Git branch ([exercises/module-04](#)) or in an `exercises/` folder excluded from `dbt_project.yml` model paths. **Do not put them in the active model path.** Model 3 contains a Jinja Parse error (`{{ if }}`/`{{ endif }}`), which prevents `dbt compile` from running at all if the file is parsed as a live model.

Suggested setup:

```
git checkout -b exercises/module-04
# place exercise files in exercises/module04/
# confirm model-paths in dbt_project.yml does NOT include exercises/
git push origin exercises/module-04
```

Participants:

```
git checkout exercises/module-04
dbt build # → Parse error from Model 3 – that's the starting point
```

The 3 buggy models

Model 1 — `int_pipeline_stages.sql`

```
{{ config(materialized='table') }}
SELECT pipeline_stage_id, stage_name, is_closed
FROM {{ source('hubspot', 'pipeline_stages') }}
```

Model 2 — `fct_daily_ticket_volume.sql` (annotated: 50M rows/day)

```
{{ config(materialized='table') }}
SELECT ticket_date, COUNT(*) AS ticket_count
FROM {{ ref('dim_ticket') }} GROUP BY 1
```

Model 3 — `dim_patient.sql`

```
{{ config(materialized='incremental', unique_key='patient_key',
on_schema_change='ignore') }}
SELECT patient_key, email, updated_at FROM {{ ref('stg_hubspot__contacts') }}
{{ if is_incremental() }} WHERE updated_at > (SELECT MAX(updated_at) FROM {{ this }})
{{ endif }}
```

All 5 bugs with fixes

#	Model	Bug	Type	Fix
1	Model 1	<code>materialized='table'</code> on a staging/intermediate model	Silent (convention violation)	Change to <code>materialized='view'</code>
2	Model 2	<code>materialized='table'</code> on a 50M-row/day fact table	Silent (cost)	Change to <code>incremental</code> with <code>unique_key</code> + <code>on_schema_change</code> + <code>{{ if is_incremental() }}</code> filter
3	Model 3	<code>on_schema_change='ignore'</code>	Silent (data)	Change to <code>on_schema_change='sync_all_columns'</code>
4	Model 3	<code>{{ if is_incremental() }}</code> — wrong delimiter	Compile error (Parse)	Change to <code>{{% if is_incremental() %}}</code>
5	Model 3	<code>{{ endif }}</code> — wrong delimiter	Compile error (Parse)	Change to <code>{{% endif %}}</code>

Key teaching point: Bugs 4 and 5 cause immediate failure — dbt raises a Parse error and tells participants exactly where. Bugs 1, 2, and 3 are silent: the project runs, builds succeed, but the output is wrong or expensive. The dangerous bugs are the ones dbt doesn't catch.

Expected solutions

Model 1 — fixed:

```
{{ config(materialized='view') }}

SELECT pipeline_stage_id,
       stage_name,
       is_closed
FROM {{ source('hubspot', 'pipeline_stages') }}
```

Model 2 — fixed:

```
{{ config(
    materialized='incremental',
    unique_key='ticket_date',
    on_schema_change='sync_all_columns'
) }}
SELECT ticket_date,
       COUNT(*) AS ticket_count
FROM {{ ref('dim_ticket') }}
{% if is_incremental() %}
WHERE ticket_date > (SELECT MAX(ticket_date) FROM {{ this }})
{% endif %}
GROUP BY 1
```

Model 3 — fixed:

```
{{ config(
    materialized='incremental',
    unique_key='contact_key',
    on_schema_change='sync_all_columns'
) }}
SELECT contact_key,
       email,
       updated_at
FROM {{ ref('stg_hubspot__contacts') }}
{% if is_incremental() %}
WHERE updated_at > (SELECT MAX(updated_at) FROM {{ this }})
{% endif %}
```

Common mistakes

Mistake	Guidance
Fixes only Bug 4/5 (compile errors) and misses the silent bugs	Ask: "The project now runs — are you done? Check the config carefully."

Mistake	Guidance
Doesn't know why Model 1 must be a view	"What does staging do? It renames and casts. No logic, no storage needed."
Converts Model 2 to incremental but forgets <code>{% if is_incremental() %}</code>	"Without the filter, every run is a full scan — same cost as a table."
Fixes <code>on_schema_change</code> in Model 3 but doesn't explain the risk	Push for explanation: "What would happen in production if this stayed as <code>ignore</code> ?"
Confuses <code>{{ }}</code> (expression) with <code>{% %}</code> (statement)	Draw the distinction: <code>{{ }}</code> renders a value in SQL; <code>{% %}</code> controls execution flow — it produces nothing in the output

Debrief order

1. **Start with Model 3, Bugs B/C** — the compile errors. Most participants find these first via the dbt error message. Confirm everyone understands *why* `{% %}` is required for control flow.
2. **Model 3, Bug A** — `on_schema_change='ignore'`. Ask: "Would dbt have warned you?" → No. Ask: "When would you notice?" → Only when a column goes missing downstream, possibly weeks later.
3. **Model 1** — straightforward convention violation. Reinforce: staging is always a view; CI will enforce this.
4. **Model 2** — the cost argument. Ask: "How long would a full rebuild of 50M rows take every night?" Make it concrete.

Close with: "Two of five bugs crashed the build immediately. Three ran quietly and produced wrong results or wasted money. Which category is more dangerous?"

Verify success

Participants should be able to state the fix *and* the reason for each bug. Running is secondary — the goal is code reading and materialization reasoning.

Module 05 — Add Sources and Complete the Staging Layer (25 min)

What participants do

1. Add `owners` to `sources.yml` with freshness config
2. Write `stg_hubspot_owners.sql`
3. Compile and verify the source path
4. Run all staging and check freshness

Expected outcome: Task 1

The `sources.yml` should now have three tables under `hubspot`:

```
version: 2

sources:
  - name: hubspot
    database: BRONZE
    schema: HUBSPOT
    tables:
```

```

- name: contacts
- name: deals
- name: owners
  loaded_at_field: _loaded_at
  freshness:
    warn_after: {count: 14, period: hour}
    error_after: {count: 25, period: hour}

```

Note: `contacts` and `deals` don't need freshness config for this exercise. Only `owners` adds it here.

Expected outcome: Task 2

```

{{ config(materialized='view') }}

SELECT
  owner_id,
  first_name,
  last_name,
  email,
  _loaded_at AS loaded_at
FROM {{ source('hubspot', 'owners') }}

```

Expected outcome: Task 3

Compiled output contains: `BRONZE.HUBSPOT.owners`

Expected outcome: Task 4

```

dbt run --select staging.*    → 4 green OK rows
dbt source freshness         → shows pass/warn/error per source

```

The freshness result depends on the actual data in the training Snowflake environment. If `_loaded_at` was populated within 14 hours, it passes. Between 14–25 hours, it warns. Beyond 25 hours, it errors. In a training environment, it'll typically warn or error — that's fine and expected. The goal is to read the output, not to have it pass.

Common mistakes

Mistake	Fix
Adds <code>owners</code> outside the <code>hubspot</code> source block (wrong indentation)	YAML indentation is critical — <code>owners</code> must be under <code>tables:</code> which is under the <code>hubspot</code> source
Uses <code>loaded_at: _loaded_at</code> instead of <code>loaded_at_field</code>	The correct key is <code>loaded_at_field</code>
Forgets <code>period: hour</code> and just writes <code>count: 14</code>	Both <code>count</code> and <code>period</code> are required

Trainer notes — `+alias` configuration

+alias is an optional `dbt_project.yml` key that changes the **name of the resulting database object** without changing the filename or how `ref()` resolves it. Raise it here because sources are the first place participants realise that database names don't have to match file names.

How it works:

```
# dbt_project.yml
models:
  dbt_training:
    1_staging:
      stg_hubspot__contacts:
        +alias: stg_hubspot__contact # view in DB → stg_hubspot__contact
                                     # ref('stg_hubspot__contacts') still works
```

`ref()` uses the **node name** (= filename without `.sql`). **+alias** only changes what lands in the database.

Three use cases to explain:

1. Fix a naming mismatch — e.g. a seed file is called `raw_contacts.csv` but the source table it simulates is `contacts`. Add **+alias: contacts** so the seed lands with the right name:

```
seeds:
  dbt_training:
    raw_contacts:
      +schema: hubspot_raw
      +alias: contacts # lands as hubspot_raw.contacts – matches
source('hubspot', 'contacts')
```

This is how the exercise project works. Seeds simulate Bronze without renaming the source YAML.

2. Point to a smaller dev table — on large production tables you don't want every developer running a full-table scan in dev. Create a `contacts_dev` table with 1 000 rows and alias to it during development:

```
# profiles.yml dev target: schema = dev
seeds:
  dbt_training:
    raw_contacts:
      +alias: "{% if target.name == 'dev' %}contacts_small{% else %}contacts{% endif %}"
```

In practice this is more commonly done at the source level with environment variables, but **+alias** gives a low-friction way to demonstrate the pattern.

3. Incremental testing with two tables — to verify an incremental model handles new and changed rows correctly without waiting for a full production reload:

1. Point the source alias to `contacts_v1` (baseline data) and run `dbt run`
2. Change alias to `contacts_v2` (v1 + new rows + some updated rows) and run `dbt run --select stg_hubspot__contacts+` again

3. Observe whether the incremental model picks up only the delta

This is a quick local testing pattern. Participants can try it in Module 11 (selectors) once they're comfortable with `--select`.

Tip: `+alias` also works on seeds, snapshots, and analyses — not just models.

Module 06 — Test the Silver Layer (30 min)

What participants do

1. Create `models/silver/schema.yml` with all tests for `fct_prescription`
2. Run `dbt test --select fct_prescription` — all pass
3. Insert a duplicate row, re-run tests — one fails; read and understand the failure
4. Write `tests/assert_fct_prescription_no_zero_dosage.sql`
5. Run `dbt build --select silver.*`

Expected outcome: Task 1

File: `models/silver/schema.yml`

Seven tests on `fct_prescription`:

- `prescription_key`: unique (error), not_null (error)
- `patient_key`: not_null (error), relationships to dim_patient (error)
- `doctor_key`: not_null (error), relationships to dim_doctor (error)
- `prescription_date`: not_null (error)
- `medication_type`: accepted_values [tablet, liquid, injection, topical] (error)
- `dosage_amount`: not_null (warn)

(`notes` has no test.)

Expected outcome: Task 2

```
dbt test --select fct_prescription
```

The FK relationship tests will fail — this is intentional. `fct_prescription.sql` contains Bug 2 (see the deliberate bugs section at the top of this guide): `patient_key` and `doctor_key` aliases are swapped. The tests surface this:

```
Failure in test
relationships_fct_prescription_patient_key__patient_key__ref_dim_patient_
  Got 10 results (OWN001-style values not found in dim_patient)

Failure in test relationships_fct_prescription_doctor_key__doctor_key__ref_dim_doctor_
  Got 10 results (C001-style values not found in dim_doctor)
```

Guide participants through the debugging process:

1. Read the error — which column, which model, which expected reference?

2. Open `fct_prescription.sql` — look at the SELECT aliases
3. Compare: `p.contact_id` should produce patient-style keys (`C001`) but it's aliased as `doctor_key`
4. Fix: swap the two alias labels
5. Re-run `dbt run --select fct_prescription` then `dbt test --select fct_prescription`

After fixing the bug, 9 tests pass. The `dosage_amount not_null` shows `WARN` for rows `rx-005` and `rx-009` (those have null dosage in the training data). The pipeline continues — warn doesn't fail the build.

Expected outcome: Task 3

After inserting a duplicate `rx-001`:

```
Failure in test unique_fct_prescription_prescription_key (models/silver/schema.yml)
Got 2 results, configured to fail if != 0
```

The test SQL in the failure output looks like:

```
SELECT prescription_key, COUNT(*) AS n
FROM SILVER_DEV.TESTING__dev_yourname.fct_prescription
GROUP BY prescription_key
HAVING COUNT(*) > 1
```

After `dbt run --select fct_prescription`, the duplicate is gone and tests pass again.

Expected outcome: Task 4

File: `tests/assert_fct_prescription_no_zero_dosage.sql`

```
SELECT
    prescription_key,
    dosage_amount
FROM {{ ref('fct_prescription') }}
WHERE dosage_amount = 0
```

Running this against the clean training data returns **0 rows** → test passes. (No prescription in the training data has `dosage_amount = 0`.)

Expected outcome: Task 5

```
dbt build --select silver.*
```

Execution order (DAG):

1. `dim_patient` → tests
2. `dim_doctor` → tests
3. `dim_pipeline` → tests (none yet — no schema.yml entry for `dim_pipeline`)

4. `fct_prescription` → tests (after `dim_patient` and `dim_doctor` confirm clean)

Common mistakes

Mistake	Fix
Writes <code>severity: error</code> at the top of the column block, not inside <code>config</code> :	Correct: - <code>unique:\n config:\n severity: error</code>
Uses <code>ref('dim_patient')</code> without quotes inside <code>to</code> :	Must be <code>to: ref('dim_patient')</code> — the function call is the value
Singular test uses <code>HAVING COUNT(*) > 0</code>	The simple <code>WHERE dosage_amount = 0</code> pattern is correct — any returned row fails the test, no aggregation needed
Forgets <code>version: 2</code> header in <code>schema.yml</code>	dbt 1.5+ doesn't require it but it's good practice and avoids confusion

Module 07 — Document the Silver Layer (20 min)

What participants do

1. Add description + grain statement to `fct_prescription` in `schema.yml`
2. Add full documentation for `dim_pipeline` (grain + 7 column descriptions + tests)
3. Run `dbt docs generate` & `dbt docs serve` and navigate the docs site

Expected outcome: Task 1

The `fct_prescription` description must include a grain statement. Accept any wording that clearly states: **one row = one prescription record**.

Acceptable: *"Grain: one row per prescription event, identified by prescription_key."*

Not acceptable: *"This model contains prescriptions."* (No grain statement.)

Column descriptions: any clear, accurate prose. The content matters more than exact phrasing.

Expected outcome: Task 2

The `dim_pipeline` grain statement must explain the SCD2 pattern. Accept any wording that states: **one row per pipeline per validity period** and that `dbt_valid_to IS NULL` marks the current version.

The `dbt_valid_to` column description must mention that `NULL` means current — this is the most commonly missed detail.

Tests on `dim_pipeline`:

- `pipeline_key`: unique + not_null (error)
- `is_current`: not_null (error)

(No test required on `hubspot_pipeline_id` for this exercise — it's not unique in an SCD2 table.)

Expected outcome: Task 3

```
dbt docs generate
dbt docs serve
```

Browser opens at <http://localhost:8080>. Participants should be able to:

1. Find `fct_prescription` → see their grain statement in the description panel
2. Find `dim_pipeline` → click the lineage DAG tab → see Bronze sources upstream
3. Click `dbt_valid_to` in the column list → see their description

Common mistakes

Mistake	Fix
Writes a description for <code>dim_pipeline</code> but forgets the grain statement	A grain statement is mandatory for Silver models — the CI check fails without it
Adds <code>hubspot_pipeline_id</code> as unique test	It's NOT unique in SCD2 — <code>hs-pipeline-003</code> appears twice. Point them to <code>data/silver/dim_pipeline.csv</code>
<code>dbt docs serve</code> shows blank or 404	You must run <code>dbt docs generate</code> first — the serve command reads the generated artifact

Verify success (trainer check)

After `dbt docs generate`, the `catalog.json` in `target/` contains column descriptions. You can grep for the grain statement:

```
grep -l "one row per prescription" target/catalog.json
```

End of Tier 1 — Debrief Checklist

Run through this list with participants. Everyone should be able to answer **from memory**:

- ☐ What is the difference between `{{ source() }}` and `{{ ref() }}`?
- ☐ Why is staging always a view?
- ☐ Where does `profiles.yml` live, and why is it not in the repo?
- ☐ What does `dbt build` do differently from `dbt run` & `dbt test`?
- ☐ What is a grain statement, and where does it go?
- ☐ What is the difference between `error` and `warn` severity? Give an example of each.
- ☐ What does the `relationships` test check?
- ☐ What is `dbt_valid_to` in an SCD2 model?

If anyone can't answer items 1–4, they're not ready for Tier 2. Recommend they redo Module 03 and Module 06 exercises independently before the next session.

Data Reference

All training data lives in `data/`. It's consistent — all foreign keys resolve, no invalid `medication_type` values, no zero dosages, and the SCD2 example clearly shows a pipeline rename.

Key values to know for demos and debugging:

Table	Notable rows
<code>fct_prescription</code>	<code>rx-005</code> , <code>rx-009</code> have null <code>dosage_amount</code> — the warn test fires on these
<code>fct_prescription</code>	No row has <code>dosage_amount = 0</code> — singular test passes clean
<code>dim_pipeline</code>	<code>hs-pipeline-003</code> has two rows — <code>pipeline_key = 3</code> (historical, <code>is_current = false</code>) and <code>pipeline_key = 4</code> (current)
<code>hubspot_contacts</code>	<code>C006</code> , <code>C007</code> belong to <code>hs-pipeline-003</code> — will get two rows if joined to <code>dim_pipeline</code> without <code>WHERE is_current = true</code>

Use the `hs-pipeline-003` double-join scenario as a live demo of why `is_current = true` is always required when joining SCD2 dimensions.

Tier 2 — Working Effectively (Modules 08–12)

The exercise project targets **Snowflake** during training. For local testing without Snowflake credentials, use `-target test` which runs against DuckDB. Python ≤ 3.12 is required (dbt-core 1.11 is not compatible with Python 3.13+).

DuckDB local testing (trainer/pre-training verification):

```
dbt seed --target test          # loads all raw_* + lookup seeds into DuckDB
dbt run --target test           # builds all models
dbt test --target test          # runs all tests (1 expected failure:
fct_prescription relationship)
```

Module 08 — Seeds and Variables (45 min)

What participants do

1. Run `dbt seed --select country_codes product_categories` to load the lookup seeds
2. Verify seed contents with `dbt show --select country_codes --limit 10`
3. Open `models/3_gold/mrt_country_summary.sql` — notice it already exists (a pre-built "colleague submission"). Query the results: all `contact_count` values are 0. Find and fix the bug.
4. Add `min_deal_amount: 0` to the `vars:` block in `dbt_project.yml` (root level, not inside `models:`)
5. Verify `fct_deal.sql` already uses `{{ var('min_deal_amount', 0) }}` — confirm with `dbt compile --select fct_deal --vars '{"min_deal_amount": 500}'`

Expected outcomes

Task	Expected result
<code>dbt seed</code>	<code>country_codes</code> (10 rows) and <code>product_categories</code> (3 rows) show OK created seed
<code>dbt show --select country_codes</code>	Rows with <code>country_code</code> , <code>country_name</code> , <code>region</code> columns; DE, AT, CH, FR etc. visible
<code>mrt_country_summary.sql</code> bug	Join reads <code>cc.country_code = p.patient_key</code> — should be <code>cc.country_code = p.country_code</code>
After bug fix + <code>dbt run --select mrt_country_summary</code>	All 10 countries show non-zero <code>contact_count</code> ; DE has the most (8)
<code>dbt compile --select fct_deal --vars '{"min_deal_amount": 500}'</code>	<code>target/compiled/</code> shows literal 500 in the WHERE clause

Bug 3 — `mrt_country_summary.sql` fix:

```
-- Change this line:
LEFT JOIN {{ ref('dim_patient') }} AS p ON cc.country_code = p.patient_key -- BUG

-- To this:
LEFT JOIN {{ ref('dim_patient') }} AS p ON cc.country_code = p.country_code -- FIXED
```

Note: The pre-built solution references `dim_patient` (not `dim_contact`). This is intentional — participants see the correct domain name from the start. The only bug is the wrong join column.

Expected `dbt_project.yml` change: The `vars:` key goes at the root level of the file, parallel to `name:` and `models:`. If placed inside the `models:` block, `dbt run` fails with a parse error.

Common mistakes

Mistake	Fix
Places <code>vars:</code> inside the <code>models:</code> block	It must be at root level — show the before/after snippet from the lesson plan
Forgets to add <code>min_deal_amount</code> to <code>dbt_project.yml</code> and only edits <code>fct_deal.sql</code>	Without the <code>vars:</code> default, running without <code>-vars</code> raises "variable not defined"
Hardcodes <code>BRONZE.HUBSPOT.contacts</code> in <code>mrt_country_summary</code> instead of <code>ref('country_codes')</code>	Seeds are referenced with <code>{{ ref() }}</code> exactly like models
Uses <code>materialized='view'</code> for the Gold mart	Gold convention is <code>table</code> ; discuss cost implications of views over large joins
Joins on the wrong column (e.g., <code>country_name</code> instead of <code>country_code</code>)	Point to the seed CSV — the join key is <code>country_code</code>

Verify success

```
dbt seed
dbt run --select mrt_country_summary
dbt compile --select fct_deal --vars '{"min_deal_amount": 500}'
```

Inspect `target/compiled/.../models/silver/fct_deal.sql` — confirm `500` appears in the WHERE clause.

Bonus challenge expected answer

Participants scan staging and Silver models for hardcoded lookup values. Common correct answers include:

- A CASE WHEN on `pipeline_stage` labels in `stg_hubspot__contacts` → seed: `pipeline_stages.csv` with `stage_id`, `stage_name`, `is_closed`
- A CASE WHEN on `medication_type` in `fct_prescription` → seed: `medication_types.csv` with `type_code`, `label`

Accept any answer that (a) identifies a real model and column, (b) shows valid CSV column names, and (c) rewrites the model to use `{{ ref('seed_name') }}`. The goal is the reasoning process, not a specific right answer.

Module 09 — Jinja and Macros (90 min)

What participants do

1. Create `macros/safe_cast.sql` with three parameters: `column_name`, `target_type`, `fallback=None`
2. Verify the macro compiles: `dbt compile --select stg_hubspot__deals`
3. Replace the raw `amount` column in `stg_hubspot__deals.sql` with `{{ safe_cast('amount', 'DOUBLE') }}`
4. Run `dbt build --select stg_hubspot__deals` and confirm it passes
5. Open `target/compiled/` and confirm `amount` compiles to `TRY_CAST(amount AS DOUBLE)` (no Jinja in output)
6. Add `{{ dbt_utils.generate_surrogate_key(['deal_id']) }}` AS `deal_key` to `fct_deal.sql` source CTE
7. Run `dbt build --select fct_deal` and open compiled SQL to confirm `deal_key` compiles to an MD5 expression

Expected outcomes

Task	Expected result
<code>macros/safe_cast.sql</code> exists	File compiles without error; <code>dbt ls</code> shows no warnings
<code>dbt compile --select stg_hubspot__deals</code>	<code>target/compiled/</code> shows <code>TRY_CAST(amount AS DOUBLE)</code> — no Jinja in output
<code>dbt build --select stg_hubspot__deals</code>	Green — model and all tests pass
<code>dbt build --select fct_deal</code>	Green — <code>deal_key</code> column present, compiles to <code>MD5(CAST(deal_id AS TEXT))</code> in DuckDB

Expected `safe_cast.sql`:

```
{% macro safe_cast(column_name, target_type, fallback=None) %}
  {%- if fallback is not none -%}
    COALESCE(TRY_CAST({{ column_name }} AS {{ target_type }}), {{ fallback }})
  {%- else -%}
    TRY_CAST({{ column_name }} AS {{ target_type }})
  {%- endif -%}
{% endmacro %}
```

Note: The `-` whitespace-stripping inside `{%- -%}` is optional but good practice — without it, extra blank lines appear in compiled output.

Expected `fct_deal.sql` source CTE:

```
WITH source AS (
  SELECT
    {{ dbt_utils.generate_surrogate_key(['deal_id']) }} AS deal_key,
    deal_id,
    deal_name,
    pipeline_id,
    amount,
    expected_close_date,
    ingested_at
  FROM {{ ref('stg_hubspot_deals') }}
)
SELECT * FROM source
```

Common mistakes

Mistake	Fix
Uses <code>{{ macro safe_cast(...) }}</code> instead of <code>{% macro safe_cast(...) %}</code>	<code>{{ }}</code> is an expression delimiter (outputs a value); <code>{% %}</code> is a statement delimiter (defines structure). Using <code>{{ }}</code> for macro definition causes an immediate parse error
Forgets <code>{% endmacro %}</code>	dbt raises a parse error — the block is never closed
Uses <code>CAST()</code> instead of <code>TRY_CAST()</code>	<code>CAST()</code> raises an error on type mismatch; <code>TRY_CAST()</code> returns NULL — the whole point of the macro
Doesn't run <code>dbt deps</code> before Task 3	<code>dbt_utils</code> must be installed first — <code>dbt deps</code> downloads it to <code>dbt_packages/</code>
Puts <code>generate_surrogate_key</code> in a macro instead of in the model CTE	Surrogate key generation is structural identity — it belongs in the model's source CTE, not in a shared abstraction
Calls <code>{{ safe_cast(amount, 'DOUBLE') }}</code> (unquoted column name)	The column name must be a string argument: <code>{{ safe_cast('amount', 'DOUBLE') }}</code>

Verify success

```
dbt compile --select stg_hubspot__deals
# open target/compiled/.../stg_hubspot__deals.sql – confirm TRY_CAST appears, no Jinja
dbt build --select stg_hubspot__deals
dbt build --select fct_deal
# open compiled fct_deal.sql – confirm MD5 expression for deal_key
```

Bonus challenge expected answer

There is no single correct answer. Accept any response that:

1. Names a real model in the project
2. Applies the decision framework correctly (macro for structural patterns, SQL CTE for business logic)
3. If recommending a macro: states a clear name, parameter list, and explains why it's structural not business logic
4. If recommending against: explains what makes it better left as SQL (single-use, business meaning, testability)

Common good answers: the `updated_at > MAX(updated_at)` incremental filter that appears in multiple models (yes, macro candidate — structural watermark pattern). The `CASE WHEN medication_type IN (...)` logic (no — business rule, should stay in SQL).

Module 10 — Slowly Changing Dimensions and Snapshots (90 min)

What participants do

1. Review `snapshots/snap_patients.sql` — it already exists as a scaffold (note the SELECT columns)
2. Run `dbt snapshot --target test` to build the initial snapshot
3. Verify: `SELECT COUNT(*) FROM snapshots.snap_patients` — all rows have `dbt_valid_to = NULL`
4. **Snowflake:** Update a patient record in the source table. **DuckDB (local testing):** Edit `seeds/raw_contacts.csv` — change contact 3's email, bump `updated_at` by one day, then run `dbt seed --target test`
5. Run `dbt snapshot --target test` again to detect the change and insert a new SCD2 row
6. Query the snapshot to confirm two rows exist for contact 3

DuckDB-specific note: In local testing mode, the snapshot reads from the seed-populated `hubspot_raw.contacts` table. After editing the CSV, participants must run `dbt seed --target test --full-refresh --select raw_contacts` before `dbt snapshot --target test`. Without this, DuckDB still holds the old data and the snapshot detects no change.

Expected outcomes

Task	Expected result
<code>snapshots/snap_patients.sql</code> exists	File has <code>{% snapshot snap_patients %} / {% endsnapshot %}</code> block; config has <code>strategy='timestamp', updated_at='updated_at', unique_key='contact_id', target_schema='snapshots'</code>
First <code>dbt snapshot</code>	Terminal shows <code>Created snapshot snapshots.snap_patients; 20 rows, all dbt_valid_to = NULL</code>

Task	Expected result
<code>SELECT COUNT(*) FROM snapshots.snap_patients</code>	20 rows (one per contact)
After update + snapshot cycle	Contact 3 has exactly two rows: original email with non-NULL <code>dbt_valid_to</code> , new email with <code>dbt_valid_to = NULL</code>
All other contacts	Still one row each; <code>dbt_valid_to = NULL</code> unchanged

Expected `snap_patients.sql` (already pre-built in scaffold — participants review and run it):

```
{% snapshot snap_patients %}

{{
    config(
        target_schema = 'snapshots',
        unique_key     = 'contact_id',
        strategy       = 'timestamp',
        updated_at      = 'updated_at'
    )
}}

SELECT
    contact_id,
    first_name,
    last_name,
    email,
    country_code,
    pipeline_stage_id,
    created_at,
    updated_at
FROM {{ ref('stg_hubspot__contacts') }}

{% endsnapshot %}
```

Expected verification query for Task 5:

```
SELECT contact_id, email, dbt_valid_from, dbt_valid_to
FROM snapshots.snap_patients
WHERE contact_id = 3
ORDER BY dbt_valid_from;
```

Expected: two rows. Row 1: original email, `dbt_valid_to` = bumped timestamp. Row 2: new email, `dbt_valid_to = NULL`.

Common mistakes

Mistake	Fix
Runs <code>dbt snapshot</code> without reloading data first	In Snowflake: UPDATE the source table. In DuckDB: edit CSV + <code>dbt seed --full-refresh</code> . Without this, no change is detected
Uses <code>{% snapshot %}</code> with <code>{{ config() }}</code> on the same line as the <code>{% snapshot %}</code> tag	The <code>{{ config() }}</code> block goes on its own line inside the snapshot, not on the opening tag line
Sets <code>strategy = 'check'</code> instead of <code>'timestamp'</code>	Valid approach, but slower and not the intended exercise. Redirect to <code>timestamp</code> and explain why it's preferred when <code>updated_at</code> is reliable
Bumps <code>updated_at</code> but doesn't change the email	The snapshot still detects the change (any difference in <code>updated_at</code> triggers a new row) — good teaching moment: timestamp strategy is driven by timestamp, not column content
Forgets <code>OR dbt_valid_to IS NULL</code> in the bonus query	The current row has no end date. Without this condition, current-state rows are excluded — a classic SCD2 bug

Verify success

```
dbt snapshot --target test
# query: SELECT COUNT(*) FROM snapshots.snap_patients -- expect 20 rows, all
dbt_valid_to = NULL

# after data update:
dbt seed --target test --full-refresh --select raw_contacts
dbt snapshot --target test
# query: SELECT * FROM snapshots.snap_patients WHERE contact_id = 3 ORDER BY
dbt_valid_from
# expect: 2 rows
```

Bonus challenge expected answer

```
SELECT
    contact_id,
    email
FROM snapshots.snap_patients
WHERE contact_id = 3
    AND dbt_valid_from <= '2024-06-01'
    AND (dbt_valid_to > '2024-06-01' OR dbt_valid_to IS NULL);
```

Returns exactly one row: the version active on 2024-06-01. Either email can be correct depending on when the edit was applied — the important thing is the `OR dbt_valid_to IS NULL` guard.

Module 11 — Selectors, Tags, and Running Subsets (45 min)

What participants do

5 written scenarios: participants write the exact `dbt` command for each. No dbt project execution required — the exercise is command syntax.

1. Rebuild only Silver fact models and their tests
2. Run `fct_prescription` and all upstream dependencies
3. Run `mrt_deals_funnel` and everything downstream
4. Run all daily-tagged models, excluding Gold marts
5. After a partial failure, rerun only the models that errored

Bonus: add a `finance` tag to two models and write the selection command.

Expected outcomes

Scenario	Correct command
1 — Silver facts + tests	<code>dbt build --select silver.facts.*</code> or <code>dbt build --select tag:daily</code>
2 — <code>fct_prescription</code> + upstream	<code>dbt run --select +fct_prescription</code>
3 — <code>mrt_deals_funnel</code> + downstream	<code>dbt run --select mrt_deals_funnel+</code>
4 — Daily models, exclude Gold	<code>dbt build --select tag:daily --exclude tag:weekly</code> or <code>--exclude gold.*</code>
5 — Rerun failures only	<code>dbt run --select result:error --state ./target</code>

Bonus expected answer:

```
# dbt_project.yml
models:
  analytics:
    silver:
      facts:
        fct_deal:
          +tags: ['daily', 'finance']
    gold:
      mrt_deals_funnel:
        +tags: ['weekly', 'finance']
```

Selection command:

```
dbt build --select tag:finance
```

Common mistakes

Mistake	Fix
Uses <code>dbt run</code> for scenarios 1 and 4 instead of <code>dbt build</code>	The scenarios explicitly ask for tests to run — only <code>dbt build</code> runs models and tests together
Writes <code>+fct_prescription+</code> for scenario 2	The trailing <code>+</code> adds downstream dependents; the scenario asks for upstream only — correct answer is <code>+fct_prescription</code> (prefix only)
Writes <code>fct_prescription+</code> for scenario 2	That selects downstream, not upstream. The <code>+</code> is on the wrong side. Draw the DAG direction to clarify
Writes <code>--state ./prod-artifacts</code> for scenario 5	In scenario 5 the run was local and the state is in <code>./target</code> . The <code>--state</code> path depends on where the previous run's <code>manifest.json</code> lives
Adds <code>finance</code> tag in wrong YAML nesting	Show the correct nesting depth in <code>dbt_project.yml</code> — the tag must be under the specific model name, not at the <code>silver:</code> level

Verify success

```
dbt ls --select +fct_prescription
# confirm: stg_hubspot__contacts, stg_hubspot__owners, stg_prescriptions, dim_patient,
dim_doctor, dim_product, fct_prescription

dbt ls --select tag:daily
# confirm: fct_deal, fct_prescription

dbt ls --select silver.facts.*
# confirm: same two models
```

Module 12 — CI/CD and Slim CI (90 min)

Note: This module has no dbt exercise project tasks. The exercise is a code review of a broken GitHub Actions YAML — no `dbt build` commands are run. Participants work through the YAML analysis on paper or in a text editor.

What participants do

1. Read a broken GitHub Actions YAML and identify all 3 bugs
2. For each bug: explain what it is, why it is a problem, and write the fix
3. Write the exact `dbt build` command for a specific CI scenario
4. Answer three one-sentence questions about `--defer`

Expected outcomes

Task 1 — Three bugs in the broken YAML:

Bug	Location	Problem	Fix
Bug 1	<code>SNOWFLAKE_PASSWORD:</code> <code>"Snowflake_Prod_2024_v3"</code>	Hardcoded credential in a committed file — visible to all	<code>SNOWFLAKE_PASSWORD: \${{ secrets.SNOWFLAKE_PASSWORD }}</code>

Bug	Location	Problem	Fix
		repo members; fails GitHub secret scanning	
Bug 2	<code>dbt run \</code> in the build step	<code>dbt run</code> builds models but does not run tests — the core CI guarantee is broken silently	Change to <code>dbt build \</code>
Bug 3	<code>--state ./target \</code>	<code>./target</code> is the local output of the <i>current</i> run (empty at start); the production manifest was downloaded to <code>./prod-artifacts</code> — wrong path causes <code>state:modified+</code> to fail or treat all models as modified	Change to <code>--state ./prod-artifacts \</code>

Task 2 — Correct `dbt build` command for the CI scenario:

```
dbt build \
  --select state:modified+ \
  --defer \
  --state ./prod-artifacts \
  --target ci
```

Task 3 — `--defer` one-sentence answers (accept paraphrases):

1. `--defer` tells dbt to read the production relation for any model not in the current selection set, instead of rebuilding it in CI.
2. This matters because without it, a change to a single Gold mart would force CI to rebuild every upstream staging and dimension model just to have input data.
3. Without `--defer`, CI would rebuild all 15+ upstream models before it could even reach the changed mart — defeating the cost savings of Slim CI.

Common mistakes

Mistake	Guidance
Finds bugs 2 and 3 but misses bug 1	Ask: "What happens if a contractor forks this repo and reads the YAML?" — the hardcoded password is the most serious bug even if it's not the most technically obvious
Explains bug 3 as "the manifest doesn't exist" rather than "wrong path"	Both are valid descriptions; probe for understanding: "Where did the workflow download the manifest to?"
Task 2: omits <code>--defer</code>	Ask: "If you only select <code>state:modified+</code> , what happens when dbt tries to read <code>stg_hubspot_deals</code> — does it rebuild it?" — yes, without <code>--defer</code> it would try to rebuild all upstream

Mistake	Guidance
Task 3: conflates <code>--defer</code> with <code>--state</code>	They are separate flags: <code>--state</code> provides the manifest for comparison; <code>--defer</code> controls which relations are read from production at runtime
Claims CI writes to PROD	CI writes to <code>DBT_CI_<run-id></code> — a unique throwaway schema. Ask: "Which target does the <code>slim_ci</code> job use?" → <code>ci</code> target, never <code>prod</code>

Verify success

Participants should be able to state all three bugs with fixes and explain the reasoning. No `dbt` commands are run for this exercise. The trainer check is whether each participant can explain:

- Why hardcoded secrets are a security risk (not just "it's bad practice")
- Why `dbt build` is required (not `dbt run`) for CI to have any value
- Why `--state ./target` is wrong (the production manifest is in `./prod-artifacts`)

End of Tier 2 — Debrief Checklist

Run through this list with participants. Everyone should be able to answer **from memory**:

- ☐ What is the difference between a lookup seed and a development seed?
- ☐ When is a piece of data better managed as a seed versus a source?
- ☐ What is the difference between `{{ }}` and `{% %}` in Jinja? Give an example of each in a macro.
- ☐ What does a macro compile to — is there any Jinja in the final SQL sent to the warehouse?
- ☐ Why does `dbt native --full-refresh` break downstream foreign keys when using native snapshots?
- ☐ What does `dbt_valid_to = NULL` mean in a snapshot table?
- ☐ What does `state:modified+` select in a Slim CI build?
- ☐ Why is `dbt build` required in CI and not `dbt run`?
- ☐ What is `--defer`, and what problem does it solve for CI compute cost?

If anyone cannot answer items 7–9, they are not ready for Tier 3. Recommend they re-read Modules 11 and 12 and attempt the exercises independently before the next session.